

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Gvinay Kumar Reddy	Reg. No.:	192472093
Section / Batch:	CSE AI	Date:	06/08/2026

Code of Conduct

I Gvinay Kumar Reddy certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Gvinay Kumar Reddy

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6

Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7
--	---	----

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation. Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset. Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents. from

```

nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())

```

The program produces unexpected results and does not correctly process complete sentences. Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.

5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.
7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words. Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features: from

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
stemmer = PorterStemmer()
documents = [
    "connected connection connecting connectivity", "studies
    studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)
features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.
5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

1. Identification of Incorrect Morphological Analyses

The main problem is that the system treats prefixes and suffixes as removable characters instead of meaningful morphological units. A correct analysis must identify the root and distinguish derivational and inflectional affixes.

Word	Correct Decomposition	Affix Type
treatment	treat + ment	-ment derivational
treatable	treat + able	-able derivational
retreatment	re + treat + ment	re-, -ment derivational
treated	treat + ed	-ed inflectional
untreated	un + treat + ed	un- derivational, -ed inflectional

2. Morphological Decomposition

Treatment = treat + ment. The root is treat and -ment changes the verb into a noun, so it is derivational.

Treatable = treat + able. The suffix -able creates an adjective meaning capable of being treated; it is derivational.

Retreatment = re + treat + ment. The prefix re- means again and -ment forms a noun. Both are derivational.

Treated = treat + ed. The suffix -ed marks past tense or past participle and is inflectional.

Untreated = un + treat + ed. The prefix un- expresses negation and is derivational, while -ed is inflectional.

3. Root Cause of the Error

- Derivational affixes carry semantic information.
- Inflectional affixes carry grammatical information.
- Multiple affixes may occur in one word.
- Removing un- from untreated changes its meaning.
- Removing re- from retreatment loses the meaning 'again'.
- Treating -ment as an inflection can incorrectly merge nouns and verbs.

Question 1 – Continued

4. Corrected Morphological Strategy

A better pipeline identifies the word, separates its affixes, finds the root, classifies each affix, and stores the complete morphological representation.

Input Word



Tokenization



Affix Identification



Root Identification



Derivational / Inflectional Classification



Morphological Representation

For example, untreated is represented as un + treat + ed, where un- is DERIVATIONAL, treat is the ROOT, and -ed is INFLECTIONAL. The system should store this information rather than simply deleting affixes.

5. Simple Python Implementation

```
words = ["treatment", "treatable", "retreatment",  
        "treated", "untreated"]
```

```
analysis = {  
    "treatment": ["treat", "ment"],  
    "treatable": ["treat", "able"],  
    "retreatment": ["re", "treat", "ment"],  
    "treated": ["treat", "ed"],  
    "untreated": ["un", "treat", "ed"]  
}
```

```
types = {  
    "ment": "Derivational",  
    "able": "Derivational",  
    "re": "Derivational",  
    "ed": "Inflectional",  
    "un": "Derivational"  
}
```

for word in words:

```
    print(word, "->", analysis[word])
```

Expected output:

treatment -> ['treat', 'ment']

treatable -> ['treat', 'able']

retreatment -> ['re', 'treat', 'ment']

treated -> ['treat', 'ed']

untreated -> ['un', 'treat', 'ed']

6. Original vs Corrected Analysis

Word	Original Faulty Approach	Corrected Analysis
treatment	treat	treat + ment
treatable	treat	treat + able
retreatment	treatment	re + treat + ment
treated	treat	treat + ed
untreated	treated	un + treat + ed

7. Effect on Biomedical Information Retrieval

Correct morphological analysis improves biomedical information retrieval because it preserves meaning while identifying common roots. Words such as treat, treatment, treated, retreatment and untreated can be represented separately while still being connected through the root treat.

- Reduces false matches

- **Reduces loss of biomedical meaning**
- **Reduces incorrect indexing**
- **Reduces semantic ambiguity**
- **Can improve precision and recall**

Question 2 – Error Analysis of Finite-State Morphological Parsing

1. Correct Morphological Analyses

Word	Prefix	Root	Suffix
replayed	re-	play	-ed
unhappier	un-	happy	-er
disconnected	dis-	connect	-ed
players	—	play	-er + -s
restarting	re-	start	-ing
unreadable	un-	read	-able

The important issue is that the parser must support more than one affix. A word may contain multiple suffixes, such as **players = play + er + s**.

2. Incorrect FSA/FST Structure

START → PREFIX → ROOT → SUFFIX → END

This faulty structure allows only one prefix and one suffix. While **player = play + er** may work, **players = play + er + s** requires two suffix transitions and can fail.

3. Corrected FST Structure

START

↓

PREFIX*

↓

ROOT

↓

SUFFIX*

↓

END

The symbol * means zero or more affixes. This allows the parser to accept multiple prefixes or suffixes where the morphology permits them.

players

START → play → er → s → END

restarting

START → re → start → ing → END

4. Python Finite-State Style Parser

```
prefixes = ["un", "re", "dis"]
```

```
suffixes = ["able", "ing", "ed", "er", "s"]
```

```
def parser(word):
```

```
    found = []
```

```
    for p in prefixes:
```

```
        if word.startswith(p):
```

```
            found.append(p + "-")
```

```
            word = word[len(p):]
```

```
            break
```

```
    root = word
```

```
    for s in sorted(suffixes, key=len, reverse=True):
```

```
        if root.endswith(s):
```

```
            found.append("-" + s)
```

```
            root = root[:-len(s)]
```

```
    for s in sorted(suffixes, key=len, reverse=True):
```

```
        if root.endswith(s):
```

```
            found.append("-" + s)
```

```
            root = root[:-len(s)]
```

```
    return found, root
```


Question 2 – Continued

4. Python Parser – Test Words

```
words = [  
    "replayed", "unhappier", "disconnected",  
    "players", "restarting", "unreadable"  
]
```

for word in words:

```
    affixes, root = parser(word)  
    print(word, ">", affixes, "+", root)
```

A production FST would use explicit states rather than Python loops, but this implementation demonstrates the required multi-affix transition logic.

5. Example FST Transitions

q0 -- re/un/dis --> q1

q0 -- ROOT -----> q2

q1 -- PREFIX -----> q1

q1 -- ROOT -----> q2

q2 -- able/ing/ed/er/s --> q3

q3 -- SUFFIX -----> q3

q3 -- END -----> ACCEPT

The transitions support re + play + ed, un + happy + er, dis + connect + ed, play + er + s, re + start + ing, and un + read + able.

Important Morphophonemic Rule

The word *unhappier* requires a spelling transformation: happy + er becomes happier. The rule is y → i before -er.

Therefore, a simple character-based FST must include spelling or morphophonemic rules.

6. Parsing Accuracy

Accuracy can be calculated using: $\text{Accuracy} = \text{Correct Analyses} / \text{Total Words} \times 100$.

```
expected = {  
    "replayed": ["re", "play", "ed"],  
    "unhappier": ["un", "happy", "er"],  
    "disconnected": ["dis", "connect", "ed"],  
    "players": ["play", "er", "s"],  
    "restarting": ["re", "start", "ing"],  
    "unreadable": ["un", "read", "able"]  
}
```

correct = 0

Compare parser output with expected output.

Increment correct when the analysis matches.

For a real experiment, execute the parser on the selected dataset and report the measured accuracy. Numerical improvement should not be claimed without actual execution.

7. Limitations and Complexity

- Unknown words are difficult to handle.
- Spelling changes require additional rules.
- Irregular morphology is difficult.
- Social-media slang creates unseen forms.
- Ambiguous morphological analyses can occur.

For a word of length n , a deterministic finite-state parser can generally process the input in approximately $O(n)$, because characters are handled through a finite number of states.

Question 3 – Error Analysis of the Stemming Program

1. Original Program

```
from nltk.stem import PorterStemmer
import pandas as pd
```

```
ps = PorterStemmer()
data = pd.read_csv("news.csv")
```

```
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

2. Programming / Preprocessing Error

The main error is `ps.stem(sentence)`. PorterStemmer is designed to process one word at a time, not a complete sentence. A sentence must first be tokenized and then each token should be stemmed separately.

Example: The players are playing football. First tokenize it into The, players, are, playing, football and punctuation. Then apply the stemmer to individual tokens.

3. Corrected Program

```
import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
```

```
nltk.download("punkt")
```

```
ps = PorterStemmer()
data = pd.read_csv("news.csv")
```

```
data["Tokens"] = data["Text"].apply(word_tokenize)
data["Stemmed"] = data["Tokens"].apply(
    lambda x: [ps.stem(w) for w in x]
)
```

```
print(data[["Text", "Tokens", "Stemmed"]].head())
```

4. AG News Dataset Version

```
!pip install datasets
```

```
from datasets import load_dataset
import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
```

```
nltk.download("punkt")
```

```
ds = load_dataset("ag_news")
data = pd.DataFrame(ds["train"][:1000])
```

```
ps = PorterStemmer()
data["Tokens"] = data["text"].apply(word_tokenize)
data["Stemmed"] = data["Tokens"].apply(
    lambda x: [ps.stem(w.lower()) for w in x]
)
```

```
print(data[["text", "Tokens", "Stemmed"]].head())
```

Question 3 – Continued

Example

Original: The players were playing matches.

Tokens: ['The', 'players', 'were', 'playing', 'matches', '.']

Stemmed: ['the', 'player', 'were', 'play', 'match', '.']

5. Twenty Problematic Stemming Cases

Word	Porter Stem	Type
studies	studi	Inflectional
studied	studi	Inflectional
studying	studi	Inflectional
players	player	Inflectional
played	play	Inflectional
playing	play	Inflectional
cars	car	Inflectional
boxes	box	Inflectional
connected	connect	Inflectional
connecting	connect	Inflectional
connection	connect	Derivational
connectivity	connect	Derivational
organization	organ	Derivational
organized	organ	Derivational
organizer	organ	Derivational
happiness	happi	Derivational
readable	readabl	Derivational
nationalization	nation	Derivational
medical	medic	Derivational
treatment	treatment	Derivational

Porter stemming is rule-based stemming rather than complete morphological analysis. Consequently, stems such as studi, happi, organ and readabl may look linguistically unnatural.

6. Why the Corrected Program Is Better

Sentence

↓

Tokenization

↓

Individual Words

↓

Stemming

↓

Stemmed Tokens

The corrected program is suitable for search, text classification, information retrieval, document clustering and NLP feature extraction because stemming is applied to individual tokens rather than to the entire sentence.

Question 4 – Error Analysis of Finite-State Morphological Parser

1. Original Program Error

```
if word.endswith("s"):
```

```
    return word[:-2], "Plural Noun"
```

This code has several errors. For cars, word[:-2] gives ca instead of car. It also assumes that every plural ends with a simple s rule.

Common Plural Patterns

Singular	Plural	Required Handling
car	cars	remove -s
box	boxes	remove -es
city	cities	-ies → -y
child	children	irregular dictionary

Irregular forms such as children, men, women, mice and feet do not follow a simple suffix rule and need special handling.

2. Corrected Parser

```
irregular = {
```

```
    "children": "child",
```

```
    "men": "man",
```

```
    "women": "woman",
```

```
    "mice": "mouse",
```

```
    "feet": "foot"
```

```
}
```

```
def parser(word):
```

```
    if word in irregular:
```

```
        return irregular[word], "Plural Noun"
```

```
    if word.endswith("ies"):
```

```
        return word[:-3] + "y", "Plural Noun"
```

```
    if word.endswith(("boxes", "classes", "buses", "watches")):
```

```
        return word[:-2], "Plural Noun"
```

```
    if word.endswith("es"):
```

```
        return word[:-2], "Plural Noun"
```

```
    if word.endswith("s"):
```

```
        return word[:-1], "Plural Noun"
```

```
    return word, "Singular"
```

Test Words and Expected Output

```
words = ["cars", "boxes", "cities", "children", "books"]
```

```
for w in words:
```

```
    print(w, "->", parser(w))
```

```
cars    -> ('car', 'Plural Noun')
```

```
boxes   -> ('box', 'Plural Noun')
```

```
cities  -> ('city', 'Plural Noun')
```

```
children -> ('child', 'Plural Noun')
```

```
books   -> ('book', 'Plural Noun')
```

Question 4 – Continued

3. WordNet Experiment

```
import nltk
```

```
nltk.download("wordnet")
```

```
from nltk.corpus import wordnet as wn
```

```
words = ["cars", "boxes", "cities", "children", "books"]
```

```
for w in words:
```

```
    synsets = wn.synsets(w, pos=wn.NOUN)
```

```
    print(w, "->", synsets[:1])
```

WordNet can be used as an external lexical resource to verify whether the words are recognized as nouns and to provide additional lexical information.

4. Original vs Corrected Output

Word	Original	Corrected
cars	ca	car
boxes	box	box
cities	cit	city
children	children / singular	child / plural
books	boo	book

The corrected parser preserves the actual morphological structure much better and handles both regular and irregular plural patterns.

5. Limitations of Rule-Based FSA

- Irregular forms require a lexicon.
- Unknown words may not match existing rules.
- Spelling changes require additional transformations.
- Ambiguous words may have multiple analyses.
- Exceptions reduce the effectiveness of fixed rules.
- Morphological productivity can generate unseen forms.
- Context-dependent interpretation cannot always be resolved by suffix rules.

For example, news ends in s but is normally a singular mass noun. Species and series can be singular or plural depending on context. Therefore, a rule-based parser should ideally be combined with a lexicon and contextual information.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

1. Original Error

Documents

↓

CountVectorizer

↓

Vocabulary / Features

↓

Stemming

This order is incorrect because CountVectorizer has already created separate vocabulary features. Stemming afterward does not rebuild the original feature matrix.

Question 5 – Continued

2. Correct Pipeline

Raw Documents



Tokenization



Lowercasing



Stemming



Normalized Documents



CountVectorizer



Feature Matrix



Classifier

3. Corrected Program

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
```

```
stemmer = PorterStemmer()
```

```
documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]
```

```
def stem_text(text):
    return " ".join(
        stemmer.stem(w) for w in text.lower().split()
    )
```

```
normalized = [stem_text(doc) for doc in documents]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(normalized)
```

```
print("Normalized documents:")
print(normalized)
print("Vocabulary:")
print(vectorizer.get_feature_names_out())
print("Feature Matrix:")
print(X.toarray())
```

4. Example Normalization

Original	Normalized
connected	connect
connection	connect
connecting	connect
connectivity	connect
studies	studi
studied	studi
studying	studi
study	studi
organize	organ
organized	organ

organizer	organ
organization	organ

5. 20 Newsgroups Experiment

The experiment compares CountVectorizer and Logistic Regression before and after stemming. The following results should be obtained by actual execution.

Important outputs after execution:

Vocabulary before = len(vec1.vocabulary_)

Vocabulary after = len(vec2.vocabulary_)

Accuracy before = acc_before

Accuracy after = acc_after

Time before = time_before

Time after = time_after

The values depend on the dataset version, number of documents, max_features, hardware, and software versions.

Therefore, invented numerical results should not be reported.

6. Comparison and Normalization

Measure	Before Correction	After Correction
Vocabulary size	Output from vec1	Output from vec2
Classification accuracy	acc_before	acc_after
Processing time	time_before	time_after
Morphological normalization	No	Yes

Overall Conclusion

The five experiments show that morphological preprocessing must be carefully designed and performed at the correct stage of an NLP pipeline. Derivational affixes should not be blindly removed because they can change meaning; inflectional affixes should be distinguished from derivational affixes; FST parsers should support multiple affixes; stemming should be token-by-token; and normalization should occur before feature extraction.

Rule-based systems are fast and interpretable but have limitations with irregular forms, spelling changes, ambiguity, exceptions and unseen words. Correct morphological preprocessing can reduce vocabulary redundancy and improve information retrieval, feature representation and NLP processing. Its effect on classification accuracy must always be verified experimentally.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	3	3	3	3	3	15	75
2	3	3	3	3	3	15	
3	3	3	3	3	3	15	
4	3	3	3	3	3	15	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Dr. Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: 06/08/2026	Date: _____	Date: _____

